UniFi Protect Device Protocol Specification

A complete protocol reference for building devices that integrate with UniFi Protect as first-party hardware. Derived from firmware reverse engineering (Ghidra), Protect application analysis (service.js), and live packet captures.

Table of Contents

1. Architecture Overview
2. Device Identity
3. UDP Discovery (Port 10001)
4. BLE Discovery
5. HTTPS Management Server
6. Adoption Flow
7. UCP4 WebSocket Connection
8. BINME Message Format
9. UCP4 Command Protocol
10. TLS Certificates
11. Device Type Registry
12. Credential Versions
13. Factory Reset and Unadoption
14. OTA Firmware Updates
15. BLE Provisioning (Advanced)
16. State Persistence (NVS)
17. Audio / Speaker System
18. Key Constants
19. Implementation Checklist

---

1. Architecture Overview

```
┌──────────────────────┐          ┌──────────────────────────────┐
│   UniFi Console      │          │     Protect Device           │
│   (Cloud Key / UDM)  │          │  (Camera, Chime, etc.)       │
│                      │          │                              │
│  ┌────────────────┐  │   UDP    │  ┌────────────────────────┐  │
│  │ Discovery      │◄─┼─10001──┼──│ Discovery Responder    │  │
│  │ Scanner        │  │          │  │ (UDP + BLE adv)        │  │
│  └────────────────┘  │          │  └────────────────────────┘  │
│                      │          │                              │
│  ┌────────────────┐  │  HTTPS   │  ┌────────────────────────┐  │
│  │ Adoption       │──┼─8080──┼─►│ HTTPS Server           │  │
│  │ Manager        │  │          │  │ /api/info, /api/adopt  │  │
│  └────────────────┘  │          │  └────────────────────────┘  │
│                      │          │                              │
│  ┌────────────────┐  │   WSS    │  ┌────────────────────────┐  │
│  │ UCP4 Server    │◄─┼─7442──┼──│ UCP4 Client (WSS)      │  │
│  │ (WebSocket)    │  │          │  │ BINME + JSON           │  │
│  └────────────────┘  │          │  └────────────────────────┘  │
└──────────────────────┘          └──────────────────────────────┘
```

Protocol stack (device -> controller):

```
┌──────────────────────────────────────┐
│         UCP4 Commands (JSON)         │
├──────────────────────────────────────┤
```

```
│      BINME Envelope (binary)        │  Two segments per frame
├─────────────────────────────────────┤
│     WebSocket (binary frames)       │  Sec-Websocket-Protocol: ucp4
├─────────────────────────────────────┤
│            TLS (mTLS)               │  Factory ECDSA cert + key
├─────────────────────────────────────┤
│              TCP                    │  To controller host:port
└─────────────────────────────────────┘
```

---

2. Device Identity

Every Protect device has a set of identity fields derived from its MAC address and product type. These are used across discovery, HTTPS, and WebSocket layers.

Field	Format	Example	Used In
`name`	Product SKU string	`"UP Chime"`	Discovery TLV 0x0C, x-type header
`sysid`	16-bit product ID	`0xF9EE`	Discovery TLV 0x10, x-sysid header
`ident` (x-ident)	`AABBCCDDEEFF`	Bare MAC hex	x-ident WSS header
`ble_name`	`Chime-AABBCCDDEEFF`	Prefixed MAC	BLE advertisement name
`anonymous_id`	`UP Chime-AABBCC`	Name + last 3 MAC bytes	Discovery TLV 0x0B
`fw_version`	Full build string	`UP.esp32.v1.7.20...`	Discovery TLV 0x03
`device_id`	UUID (from MAC or random)	`20e7c885-a9cc-0000-...`	x-device-id, TLV 0x26
`guid`	UUID (random or derived)	`caf4395d-53d6-...`	x-guid, TLV 0x2B
`mac`	6 bytes	`20:E7:C8:85:A9:CC`	TLV 0x01/0x02/0x05

Identity Construction

```
MAC = WiFi STA MAC address (6 bytes)

name         = product SKU (e.g. "UP Chime", "UVC G4 Bullet")
ident        = hex(MAC[0:6])                        # "20E7C885A9CC" (x-ident, bare MAC)
ble_name     = "<ble_prefix>-" + hex(MAC[0:6])      # "Chime-20E7C885A9CC" (BLE name only)
anonymous_id = "<name>-" + hex(MAC[3:6])            # "UP Chime-85A9CC"
sysid_hex    = uppercase hex of sysid               # "F9EE"
device_id    = UUID derived from MAC or random      # stored in NVS
guid         = separate UUID                        # stored in NVS
```

---

3. UDP Discovery (Port 10001)

The primary discovery mechanism. The UniFi Console broadcasts UDP packets to port 10001. Devices respond with a TLV-encoded identity packet.

Request

Any UDP packet to port 10001 triggers a response. The console typically sends a 4-byte probe: `01 00 00 00`. The device does NOT parse the request content.

Response Format

```
Offset  Size  Field
------  ----  -----
0x00    1     Version (always 0x01)
0x01    1     Command (always 0x00)
0x02    2     Data length (big-endian, total bytes after header)
0x04    ...   TLV fields
```

Header bytes 2-3 are the data length. The console uses this to parse the TLV chain. A value of `0x0000` will cause the console to ignore all TLV data.

TLV Entry Format

```
Offset  Size  Field
------  ----  -----
0x00    1     Type tag
0x01    2     Value length (big-endian)
0x03    N     Value data (N = length)
```

TLV Field Reference

Field IDs from the Protect application's discovery parser:

ID	Hex	Name	Size	Encoding	Description
1	0x01	HWADDR	6	bytes	Hardware MAC address
2	0x02	IPINFO	10	bytes	MAC (6B) + IPv4 (4B)
3	0x03	FWVERSION	var	string	Firmware version string
4	0x04	ADDR_ENTRY	4	bytes	IPv4 address (network byte order)
5	0x05	MAC_ADDR	6	bytes	MAC address (second copy)
10	0x0A	UPTIME	4	uint32 BE	Uptime in seconds
11	0x0B	HOSTNAME	var	string	Anonymous device name
12	0x0C	PLATFORM	var	string	Product SKU / platform name
16	0x10	SYSTEM_ID	2	uint16 **LE**	Product sysid
23	0x17	MGMT_IS_DEFAULT	1-4	uint	Default state flag (see below)
43	0x2B	GUID	16	bytes	Device GUID (binary UUID)
44	0x2C	DEFAULT_CREDS	1	uint8	Default credential version
38	0x26	DEVICE_ID	16	bytes	Device ID (binary UUID)
32	0x20	UUID_STRING	36	string	Device UUID as string
63	0x3F	SUPPORT_UCP4	var	-	UCP4 protocol support flag

Critical encoding notes:

- **SYSTEM_ID (0x10)** is read as `readUInt16LE()` — **little-endian**, not big-endian
- **MGMT_IS_DEFAULT (0x17)** logic: `isManaged = !value`

    - Value 0 → device IS managed/adopted
    - Value non-zero → device is in factory default state (adoptable)
- **UPTIME (0x0A)** is big-endian uint32 seconds

Example Response (from real UP Chime PoE, 217 bytes)

```
01 00 00 d5                                  # header: v1, cmd=0, len=213

0c 00 0c  55 50 20 43 68 69 6d 65 20 50 6f 45   # 0x0C: "UP Chime PoE"
0b 00 13  55 50 20 43 ... 30 44 42 41 36 39      # 0x0B: "UP Chime PoE-0DBA69"
0a 00 04  00 1a e4 c9                            # 0x0A: uptime = 1762505 sec
04 00 04  c0 a8 01 72                            # 0x04: IP = 192.168.1.114
02 00 0a  1c 6a 1b 0d ba 69 c0 a8 01 72         # 0x02: MAC + IP
01 00 06  1c 6a 1b 0d ba 69                      # 0x01: MAC
05 00 06  1c 6a 1b 0d ba 69                      # 0x05: MAC
03 00 26  55 50 2e 65 73 70 33 32 ...            # 0x03: firmware version
10 00 02  14 ab                                  # 0x10: sysid (LE: 0xAB14)
26 00 10  [16 bytes binary UUID]                 # 0x26: device ID
17 00 01  00                                     # 0x17: is_default=0 (adopted)
2b 00 10  [16 bytes binary UUID]                 # 0x2B: GUID
20 00 24  [36 bytes UUID string]                 # 0x20: UUID string
2c 00 01  03                                     # 0x2C: credential version
```

Minimal Required TLVs

For a device to be discovered and shown as adoptable, include at minimum:

1. **0x0C** (PLATFORM) — must match a known SKU for type resolution
2. **0x0B** (HOSTNAME) — display name
3. **0x01** (HWADDR) — MAC address
4. **0x04** (ADDR_ENTRY) — IP address
5. **0x10** (SYSTEM_ID) — product sysid (little-endian)
6. **0x17** (MGMT_IS_DEFAULT) — set to non-zero for adoptable devices
7. **0x03** (FWVERSION) — firmware version
8. **0x26** (DEVICE_ID) — device UUID
9. **0x2B** (GUID) — device GUID

---

4. BLE Discovery

> ***Note:*** *This section is derived entirely from static firmware analysis (Ghidra). The BLE advertisement format, TLV contents, and byte budget have not been validated against a live controller. UDP discovery (section 3) is the tested and confirmed path.*

BLE advertisements are used as a secondary discovery mechanism, particularly for WiFi-only devices that may not yet be on the network.

Advertisement Data

BLE advertisement payload uses **manufacturer-specific data** (AD type 0xFF):

```
[Company ID: 2 bytes, little-endian]  0xC5, 0x04  (Ubiquiti: 0x04C5)
[TLV payload using same format as UDP discovery]
```

Budget: BLE advertisement is limited to 31 bytes total:

- Flags AD: 3 bytes
- Manufacturer AD header: 2 bytes

- Available for data: 26 bytes (including 2-byte company ID = 24 bytes payload)

Recommended BLE TLVs (within 24-byte budget)

TLV	Size	Content
0x0C	11B (3+8)	Platform string (`"UP Chime"`)
0x0A	7B (3+4)	IP address (4 bytes)
raw	6B	MAC address (no TLV header)
Total	**24B**	Exact fit

Note: `"UP Chime"` (8 chars) fits exactly. `"UP Chime PoE"` (12 chars) would exceed the budget — the PoE variant may need to abbreviate or omit the platform TLV from BLE and rely on UDP discovery.

Scan Response

Device name goes in the scan response (separate from advertisement data):

- AD type 0x09 (Complete Local Name)
- Value: `"Chime-AABBCCDDEEFF"` (ident string)

BLE Parameters

```
adv_int_min:  100   (62.5ms)
adv_int_max:  1000  (625ms)
adv_type:     ADV_TYPE_IND (connectable undirected)
addr_type:    BLE_ADDR_TYPE_PUBLIC
channels:     ADV_CHNL_ALL
filter:       ADV_FILTER_ALLOW_SCAN_ANY_CON_ANY
```

---

5. HTTPS Management Server

The device runs an HTTPS server on **port 8080** for the console to query info and send adoption commands. Uses the factory ECDSA certificate.

Important: Protect's `getApiPort()` returns 8080 for all non-bridge devices (8082 for BLE-B bridge).

GET /api/info

Auth: HTTP Basic (default: `ui` / `ui` for v2 devices including Chime, `ubnt` / `ubnt` for v1)

Response headers:

```
x-type: UP Chime          # Product SKU (MUST match known SKU list)
x-sysid: F9EE             # Sysid in uppercase hex
x-ident: AABBCCDDEEFF     # Bare MAC hex (NOT prefixed with "Chime-")
```

Response body (application/json):

```
{
  "name": "UP Chime",
  "fw_version": "UP.esp32.v1.7.20.0.402a5ff.240910.0648",
  "version": "v1.7.20",
  "reboot": true,
  "factoryReset": true,
```

```
  "uptime": 12345,
  "hasWifi": true,
  "hasHttpsClientOTA": true,
  "supportCustomRingtone": true,
  "featureFlags": {}
}
```

Important: The `x-type` header MUST be the exact product SKU string (e.g. `"UP Chime"`, not `"Chime"`). The Protect application uses this to resolve the device `modelKey` via `getType(x-type)`.

POST /api/adopt

Auth: Credentials in JSON body (NOT HTTP Basic auth header). The controller sends `username` / `password` fields in the POST body directly.

Request body:

```
{
  "username": "ui",
  "password": "ui",
  "hosts": ["192.168.1.1:7442"],
  "token": "abcdef1234567890",
  "protocol": "wss",
  "mode": 0,
  "nvr": "UniFi OS Console",
  "controller": "Protect",
  "consoleId": "xxxxxxxx-xxxx-xxxx-xxxx-xxxxxxxxxxxx",
  "consoleName": "My Console"
}
```

The `hosts` array contains WSS endpoints the device should connect to. Format: `"hostname:port"` (typically port 7442).

Success response: `{}` (empty JSON object, HTTP 200)

Error responses:

Status	Body	Cause
412	`"412 Precondition Failed"`	Not in adoptable state
503	`"503 Service Unavailable"`	Device busy
200	`"Missing fields"`	Required JSON fields absent
200	`"Too Many Hosts"`	hosts array > 128 entries
200	`"Token Too Long"`	Token exceeds max length
200	`"Username Too Long"`	Username exceeds max length
200	`"Password Too Long"`	Password exceeds max length
200	`"Invlaid hosts format"`	Can't parse host:port (sic — typo in firmware)
200	`"Memory Empty"`	Allocation failure
200	`"Device do not handle adopt correctly"`	Internal error

GET /api/support

Auth: HTTP Basic **Response:** `application/tar+gzip` — diagnostic bundle

---

6. Adoption Flow

Sequence Diagram

```
UniFi Console                              Device
     │                                        │
     │  ── Discovery Phase ──                 │
     │                                        │
     │  1. UDP broadcast to :10001            │
     │──────────────────────────────────────►│
     │◄──────────────────────────────────────│ TLV response (identity)
     │                                        │
     │  (and/or BLE scan finds device)        │
     │                                        │
     │  ── Identification Phase ──            │
     │                                        │
     │  2. Parse discovery TLV 0x0C           │  → getType(platform) → modelKey
     │     Parse TLV 0x17                     │  → isDefault? show "Adoptable"
     │                                        │
     │  ── User clicks "Adopt" ──             │
     │                                        │
     │  ── Adoption Phase ──                  │
     │                                        │
     │  3. POST /api/adopt (HTTPS :8080)      │
     │──────────────────────────────────────►│ Body: {hosts, token, username,
     │◄──────────────────────────────────────│       password, protocol, ...}
     │     Response: {}                       │
     │                                        │
     │  ── Connection Phase ──                │
     │                                        │
     │  4. Device connects WSS                │
     │◄──────────────────────────────────────│ WSS to host:port from adopt
     │     Upgrade with identity headers      │ x-type, x-ident, x-token, etc.
     │                                        │
     │  5. Console verifies WSS client        │
     │     - Check x-ident (MAC)              │
     │     - Resolve modelKey from x-type     │
     │     - Validate x-token                 │
     │     - Check TLS fingerprint            │
     │                                        │
     │  ── Session Phase ──                   │
     │                                        │
     │  6. getConsoleInfo (device → console)   │
     │◄──────────────────────────────────────│ BINME {type:"request", action:"getConsoleInfo"}
     │──────────────────────────────────────►│ BINME {type:"response"} + {consoleId}
     │                                        │
     │  7. Controller command sequence        │
     │──────────────────────────────────────►│ getInfo → changeUserPassword →
     │◄──────────────────────────────────────│ networkStatus → getAudioInfo
     │                                        │
     │  8. Ongoing UCP4 session               │
     │──────────────────────────────────────►│ playSpeaker, setLEDState, etc.
     │◄──────────────────────────────────────│ Responses + statusReport (~30s)
```

WiFi State Machine (post-adoption)

After receiving the adopt payload, the device transitions through these WiFi states:

```
wifi_init (0) -> check_nvs (1) -> autolink_start (2) -> autolink_wait (3)
    -> autolink_done (4) -> sta_set_start (5) -> sta_set_done (6)
    -> ap_connecting (7) -> sta_connected (8) -> ap_connected (9)
    -> readopt (10) -> ap_get_rssi (11) -> adopted (12)
```

Adoption paths:

- `autolink` — standard discovery-based adoption
- `ble2wifi` — BLE provisioning to WiFi (secondary)

Failure handling:

- After 60 consecutive connection failures, resets to `sta_set_start` (5)
- Cycles through up to 11 hosts (modulo 11, 1-indexed)

Protect Application Internals

The Protect app dispatches adoption by `modelKey`:

modelKey	Adoption Path	Method
`aiport`	`"aiport.adopt"`	Custom handler
`sensor` (LoRa)	`"loraBridge.device.adopt"`	Via LoRa bridge
`sensor` (BLE)	`"sensor.adopt"`	Direct BLE
`linkstation`	`"linkstation.adopt"`	Custom handler
`cameraGroup`	`"cameraGroups.adopt"`	Group handler
`fob`	`"loraBridge.device.adopt"`	Via LoRa bridge
All others	`"devices.api.request"`	**HTTP POST to device**

The "all others" path (which includes camera, chime, light, viewer, speaker, siren, reader, hub, bridge) sends an HTTP POST to `https://<device-ip>:8080/api/adopt` with the management payload (port from `getApiPort()`, 8080 for non-bridge devices).

Management Payload (adopt body)

The console constructs this object and POSTs it to the device:

```
{
  "username": "<configured-username>",
  "password": "<configured-password>",
  "hosts": ["<nvr-ip>:<port>", ...],
  "token": "<random-auth-token>",
  "protocol": "wss",
  "mode": 0,
  "nvr": "<console-type>",
  "controller": "Protect",
  "consoleId": "<console-uuid>",
  "consoleName": "<console-name>"
}
```

The `hosts` array typically contains one or more `host:port` entries where the device should establish its UCP4 WebSocket connection. The standard port is **7442**.

---

7. UCP4 WebSocket Connection

After receiving the adopt payload, the device connects to the controller via WebSocket Secure (WSS).

Connection URL

```
wss://<host>:<port>
```

Where `host` and `port` come from the `hosts` array in the adopt payload.

WebSocket Upgrade Headers

The device sets `subprotocol = "ucp4"` via the **native WebSocket subprotocol field** (NOT as a custom header — see pitfall 7 below) and sends these custom headers during the WebSocket handshake:

```
x-adopted: false
x-sysid: F9EE
x-type: UP Chime
x-ident: 20E7C885A9CC
x-ip: 192.168.1.100
x-mode: 0
x-device-id: 20e7c885-a9cc-0000-0000-000000000000
x-guid: caf4395d-53d6-4a67-93fb-69c7dd879c93
x-version: UP.esp32.v1.7.20.0.402a5ff.240910.0648
x-token: abcdef1234567890
```

Header	Value	Notes
subprotocol (native)	`ucp4`	**Use native WSS subprotocol field**, not custom header
`x-adopted`	`true` or `false`	`false` for first connection
`x-sysid`	`%04X` hex	Product sysid (e.g. `F9EE`)
`x-type`	SKU string	**Must match known SKU** (e.g. `UP Chime`)
`x-ident`	Bare MAC hex	e.g. `20E7C885A9CC` — NOT prefixed with `Chime-`
`x-ip`	IPv4 string	Device's current IP
`x-mode`	`0`	Must be `"0"`
`x-device-id`	UUID string	Device unique ID
`x-guid`	UUID string	Device GUID
`x-version`	FW version string	Full firmware build string
`x-token`	Token string	From adopt payload (omit if first connection)

Console Verification (verifyUcpClient)

The Protect application validates the WebSocket connection:

1. **Rate limiting** — check connection rate per source IP

2. **Required headers** — `x-ident`, `sec-websocket-protocol`, `x-type`, `x-mode`
3. **Protocol check** — must be `"ucp4"` or `"updates"`
4. **Mode check** — must be `"0"`
5. **Type resolution** — `getDeviceManifest(sysid)` → fallback to `getType(x-type)`
6. **Device lookup** — find or create device record by MAC from `x-ident`
7. **Auth check** — if not adopted and no token: reject 403
8. **Fingerprint** — TLS client certificate fingerprint verification

TLS Configuration

```
Client certificate:  Factory ECDSA cert (secp256r1, CN=camera.ubnt.dev)
Client key:          Factory EC private key
Server verification: SKIP (skip_cert_common_name_check = true)
```

The device does NOT verify the controller's TLS certificate. The controller MAY verify the device's client certificate fingerprint.

Connection Parameters

```
Buffer size:         5120 bytes
Network timeout:     60 seconds
Reconnect timeout:   5000 ms (WSS library auto-reconnect)
Rate limit:          ~300 seconds between application-level reconnect attempts
Max host cycling:    11 hosts, modulo cycling
Max connect fails:   60 before reset to host scanning
```

WebSocket Events

Event	Value	Action
DISCONNECTED	0	Log, attempt reconnect
ERROR	1	Clear timestamps, signal error
CONNECTED	2	Transition to active state, signal success
CLOSED	4	Force reconnect state, signal

LED State During Connection

Connection State	LED State	Meaning
Init (0)	1	Startup
Connecting (1)	2	Attempting connection
Connected (2)	3	Connected to controller
Error	4	Connection error

---

8. BINME Message Format

All UCP4 messages over WebSocket use binary frames with the BINME (Binary Message Envelope) format. Each WebSocket frame contains exactly **two BINME segments**: a header segment and a body segment.

Segment Structure

```
Offset  Size  Field
------  ----  -----
0x00    1     Type       (1=request, 2=response)
0x01    1     Subtype    (hardcoded 0x01 outgoing, ignored on parse)
0x02    1     Compress   (0=raw, non-zero=zlib compressed)
0x03    1     Padding    (0x00)
0x04    4     Length     (payload size, big-endian)
0x08    N     Payload    (JSON string, or zlib-compressed JSON)
```

Frame Layout

```
┌──────────────────────────────────────────┐
│              WebSocket Binary Frame          │
├──────────────────────┬──────────────────────┤
│   Header Segment     │    Body Segment      │
│  8B header + JSON    │  8B header + JSON    │
└──────────────────────┴──────────────────────┘
```

Encoding a Message

```python
def encode_binme_message(header_json: str, body_json: str,
                         msg_type: int = 0x01) -> bytes:
    """msg_type: 0x01=request (device-initiated), 0x02=response (to controller)"""
    frame = b""
    for payload in [header_json, body_json]:
        data = payload.encode("utf-8")
        segment = bytes([
            msg_type,                     # type (0x01=request, 0x02=response)
            0x01,                         # subtype (always 0x01)
            0x00,                         # compress (raw)
            0x00,                         # padding
        ])
        segment += len(data).to_bytes(4, "big")  # length (BE)
        segment += data                           # payload
        frame += segment
    return frame
```

BINME type byte mapping:

- `0x01` (request): Used for device-initiated requests (`getConsoleInfo` , `statusReport`) AND controller-to-device commands
- `0x02` (response): Used for responses to commands (device responding to controller)

Parsing a Message

```python
def parse_binme_message(data: bytes) -> tuple[str, str]:
    segments = []
    offset = 0
    for _ in range(2):
        type_     = data[offset]
        subtype   = data[offset + 1]
        compress  = data[offset + 2]
        # padding  = data[offset + 3]
        length    = int.from_bytes(data[offset+4:offset+8], "big")
```

```
        payload   = data[offset+8:offset+8+length]
        if compress:
            payload = zlib.decompress(payload)
        segments.append(payload.decode("utf-8"))
        offset += 8 + length
    return segments[0], segments[1]  # header_json, body_json
```

Notes

- The `subtype` byte is **hardcoded to 0x01** on encode and **never checked** on parse
- Compression uses standard zlib (allocate 2x compressed size for decompression)
- Encryption (NaCl crypto_secretbox) is optional over WSS and only mandatory for BLE
- `x-mode: 0` in WebSocket headers indicates unencrypted UCP4 payloads

---

9. UCP4 Command Protocol

Message Format (Confirmed via Live Testing)

All UCP4 messages use `type` / `action` / `id` fields (NOT `METHOD` / `ID_PATH` / `requestId` as originally theorized from RE).

Request (Controller -> Device)

Header segment (BINME type=0x01):

```
{
  "type": "request",
  "action": "getInfo",
  "id": "xxxxxxxx-xxxx-xxxx-xxxx-xxxxxxxxxxxx",
  "timestamp": 1709900000
}
```

The `id` is a UUID string generated by the controller. The device must echo this exact `id` back in the response.

Body segment:

```
{ "track_no": 1, "volume": 80 }
```

Response (Device -> Controller)

Header segment (BINME type=0x02):

```
{
  "type": "response",
  "id": "xxxxxxxx-xxxx-xxxx-xxxx-xxxxxxxxxxxx",
  "responseCode": 200
}
```

The `id` is echoed from the request. `responseCode` is an integer HTTP-style status code.

Body segment:

```
{ "name": "UP Chime", ... }
```

First Message: getConsoleInfo

Immediately after WebSocket connects, the device sends:

```
// Header (BINME type=0x01):
{"type":"request","action":"getConsoleInfo","id":"dev-1","timestamp":12345}
// Body:
{}
```

Controller responds with:

```
// Header (BINME type=0x02):
{"type":"response","id":"dev-1","responseCode":200}
// Body:
{"consoleId":"xxxxxxxx-xxxx-xxxx-xxxx-xxxxxxxxxxxx"}
```

Periodic Status Report

Sent every ~30 seconds by the device:

```
// Header (BINME type=0x01):
{"type":"request","action":"statusReport","id":"dev-2","timestamp":12345}
// Body:
{
  "rssi": -45,
  "mac": "20:E7:C8:85:A9:CC",
  "ip": "192.168.1.100",
  "version": "v1.7.20",
  "uptime": 12345
}
```

Controller Command Flow (observed order)

After WSS connection and getConsoleInfo exchange, the controller sends commands in this order:

1. `getInfo` — query device identity and capabilities
2. `changeUserPassword` — rotate credentials from default (`ui` / `ui`) to random
3. `networkStatus` — query RSSI, MAC, IP, AP scan
4. `getAudioInfo` — query audio tracks and volume

Command Reference

Action	Direction	Request Body	Response Body
`getConsoleInfo`	Device → Controller	`{}`	`{consoleId}`
`statusReport`	Device → Controller	`{rssi, mac, ip, version, uptime}`	—
`getInfo`	Controller → Device	`{}`	Device info JSON
`getAudioInfo`	Controller → Device	`{}`	`{tracks, volume, speaker}`
`getSupportInfo`	Controller → Device	`{}`	Support data
`networkStatus`	Controller → Device	`{}`	`{rssi, mac, ip, apList}`
`playSpeaker`	Controller → Device	`{track_no, volume, repeat, speaker}`	`"ok"`
`playBuzzer`	Controller → Device	`{volume, freq, duration}`	`"ok"`

`setLEDState`	Controller → Device	`{led_state, repeat}`	`"ok"`
`setTimezone`	Controller → Device	timezone data	`"ok"`
`updateFirmware`	Controller → Device	`{url}`	`"ok"`
`changeUserPassword`	Controller → Device	`{username, passwordOld, passwordNew}`	`"ok"`
`changeUplinkAP`	Controller → Device	WiFi config	`"ok"`
`reboot`	Controller → Device	`{}`	`"ok"`
`factoryReset`	Controller → Device	`{}`	`"ok"`

Command Body Details

getInfo:

```
{
  "name": "UP Chime",
  "fw_version": "UP.esp32.v1.7.20.0.402a5ff.240910.0648",
  "version": "v1.7.20",
  "mac": "AA:BB:CC:DD:EE:FF",
  "uptime": 12345,
  "hasWifi": true,
  "hasHttpsClientOTA": true,
  "supportCustomRingtone": true,
  "featureFlags": {}
}
```

networkStatus:

```
{
  "rssi": -45,
  "mac": "AA:BB:CC:DD:EE:FF",
  "ip": "192.168.1.100",
  "apList": [
    {
      "essid": "NetworkName",
      "signalLevel": -45,
      "frequency": 2437,
      "quality": 65,
      "encryption": "wpa2",
      "authSuites": "psk"
    }
  ]
}
```

getAudioInfo:

```
{
  "tracks": [
    { "track_no": 0, "name": "default" },
    { "track_no": 1, "name": "custom1" }
  ],
  "volume": 80,
```

```
  "speaker": true
}
```

playSpeaker request body:

```
{ "track_no": 1, "volume": 80, "repeat": false, "speaker": true }
```

playBuzzer request body:

```
{ "volume": 100, "freq": 2000, "duration": 500 }
```

setLEDState request body (field names from RE, not verified field-by-field against live traffic)**:**

```
{
  "led_state": { "color": 2, "on_time": 1000, "off_time": 500, "pattern": "solid" },
  "repeat": true
}
```

changeUserPassword request body:

```
{ "username": "ui", "passwordOld": "ui", "passwordNew": "newpassword" }
```

Error Codes

Code	Meaning
1001	Unknown message type
1002	Unsupported action
1003	No request body
1004	Mismatched type

---

10. TLS Certificates

Factory Certificate (shared across all devices)

All Ubiquiti Protect devices ship with the same factory ECDSA certificate and private key. This is used for:

1. HTTPS server TLS (device-side)
2. WSS client mTLS (client certificate presented to controller)
3. OTA HTTPS download verification

Certificate details:

```
Algorithm:  ECDSA (secp256r1 / prime256v1)
Subject:    C=TW, L=Taipei, O=Ubiquiti Networks Inc., OU=devint, CN=camera.ubnt.dev
Email:      support@ubnt.com
Validity:   2021-05-27 to 2121-05-03
Self-signed
```

The certificate and private key can be extracted from any Ubiquiti device firmware.

Controller Certificate Handling

- The device does NOT verify the controller's TLS certificate
- `skip_cert_common_name_check = true` in the WSS client config
- The controller MAY verify the device's client cert fingerprint
- The TLS fingerprint from the initial HTTPS interaction is stored and compared against the WSS connection's client cert

Per-Device Certificate (post-adoption)

After adoption, the controller may provision a device-specific certificate:

- Device generates EC keypair + CSR locally
- Signed using CA key material provided by controller
- Stored in NVS: `certs` (0x800 bytes), `privkey` (0x200 bytes)
- Validity: 2021-01-01 to 2121-01-01

---

11. Device Type Registry

Type Resolution

The Protect application identifies devices using two methods:

1. **By sysid** — `getDeviceManifest(normalizeSysidToHex(x-sysid))` checks known sysid maps
2. **By SKU** — `getType(x-type)` matches against product SKU strings (fallback)

Chimes, and potentially other newer device types, are NOT in the sysid maps and are identified exclusively by SKU string matching.

Known Device Types

modelKey	SKU Strings	Notes
`camera`	Various (UVC G3/G4/G5/AI models)	Identified by sysid (CAMERA_TYPES_BY_SYSID)
`chime`	`"UP Chime"`, `"UP Chime PoE"`	**SKU-only** (not in sysid map)
`light`	Various UP FloodLight/Spotlight	
`viewer`	Viewport models	
`speaker`	UP Speaker, UP Speaker PoE	SPEAKER_SYSIDS
`siren`	UP Siren	SIREN_SYSIDS
`sensor`	UP Sensor (BLE or LoRa)	
`bridge`	UP Bridge	
`reader`	UA Reader models	READER_TYPES_BY_SYSID
`hub`	UA Hub	HUB_TYPES_BY_SYSID
`aiport`	AI Port models	AI_PORT_SYSIDS
`cameraGroup`	Camera group sysids	CAMERA_GROUP_SYSIDS
`linkstation`	LinkStation	LINKSTATION_SYSIDS
`fob`	Key Fob	Via LoRa bridge

Chime Platform Mapping

```
{
  "UP Chime PoE": {"platform": "esp32-d0wd", "display": "Smart PoE Chime"},
```

```
  "UP Chime":     {"platform": "esp32",      "display": "Smart WiFi Chime"}
}
```

Wireless Detection

A chime is treated as wireless if: `isChime(device) && device.featureFlags.hasWifi`

---

12. Credential Versions

The Protect application supports two default credential versions:

Version	TLV 0x2C Value	Username	Password	Used By
v1	bit1=0	`ubnt`	`ubnt`	Older cameras, some sensors
v2	bit1=1 (0x03)	`ui`	`ui`	**Chime**, newer cameras, most current devices

Resolution Order

1. Check `defaultCredentialVersion` from device firmware (discovery TLV 0x2C)
   - Value 0x03 → bit1 is set → **v2** (`ui` / `ui`)
   - Value without bit1 → **v1** (`ubnt` / `ubnt`)
2. If TLV 0x2C not present, fall back by `modelKey`

Important: The UP Chime reports cred version 0x03 (v2), so default credentials are `ui` / `ui` , NOT `ubnt` / `ubnt` .

The controller sends these credentials in the adopt POST body and uses them for HTTP Basic auth when querying `/api/info` .

---

13. Factory Reset and Unadoption

Via UCP4 (adopted devices)

The controller sends a `factoryReset` command over the UCP4 WebSocket:

```
{"type":"request","action":"factoryReset","id":"<uuid>","timestamp":...}
```

Dispatch by modelKey:

- bridge → `"bridge.resetToDefaults"`
- sensor → `"sensor.resetToDefaults"`
- aiport → `"aiport.unadopt"`
- linkstation → `"linkstation.unadopt"`
- **Default** (camera, chime, etc.) → UCP4 `factoryReset` command to device

Device-Side Reset

1. Clear NVS adoption data (hosts, token, credentials)
2. Clear provisioned certificates
3. Reset WiFi to default
4. Restart HTTPS server and discovery
5. Reboot

HTTP Endpoint

`POST /api/factoryResetWithoutWiFi` — resets adoption but preserves WiFi config.

---

14. OTA Firmware Updates

> ***Note:*** *This section is entirely from static firmware analysis. OTA has never been triggered on the ESP32 clone. The notification types (3/4), retry count, 512KB buffer size, and partition scheme details are all unverified.*

Trigger

Controller sends `updateFirmware` UCP4 command with an HTTPS URL.

Flow

1. Device receives OTA URL via UCP4
2. Sends UCP4 notification **type 3** (OTA starting)
3. Downloads firmware via `esp_https_ota()`:
   - Uses factory CA cert for server verification
   - `skip_cert_common_name_check = true`
   - HTTP buffer: 512KB
   - Up to 3 retries, 500ms between attempts
4. Sends UCP4 notification **type 4** (result: `0x01` =success, `0x00` =failure)
5. Does NOT auto-reboot — waits for separate `reboot` command

Partition Scheme

- Standard ESP32 dual OTA: `ota_0` / `ota_1` with `otadata` selector
- `esp_ota_get_next_update_partition()` selects inactive slot
- `esp_ota_set_boot_partition()` updates boot target after download

Validation

- ESP-IDF built-in: image magic byte (0xE9), segment CRC, SHA-256
- Anti-rollback compiled in but no custom version checks
- No custom signature verification beyond ESP-IDF defaults

Reboot Sequence (after OTA)

1. Controller sends `reboot` UCP4 command
2. Device stops audio playback
3. Waits 100ms
4. Calls `esp_restart()`

---

15. BLE Provisioning (Advanced)

> ***Note:*** *This section is derived entirely from static firmware analysis. The GATT service, Curve25519 handshake, and NaCl encryption have not been tested against a live controller or mobile app.*

For WiFi-only devices (like the UP Chime), BLE can be used as an alternative adoption path when the device is not yet on any WiFi network.

GATT Service

UUID	Purpose
`9280F26C-A56F-43EA-B769-D5D732E1AC67`	Primary service
`D587C47F-AC6E-4388-A31C-E6CD380BA043`	Write characteristic (unadopted)
`D085C321-807B-4AA7-BE9E-4ADA10FA5B1B`	Write characteristic (adopted)

Max write size: 500 bytes. Data fragmented if larger.

Authentication Protocol

```
Controller                          Device
     │                                   │
     │  DHPK {pubkey: 32 bytes}          │
     │──────────────────────────────────►│  Curve25519 key exchange
     │  DHPK response {pubkey: 32B}      │
     │◄──────────────────────────────────│
     │  AUTH DH {confirmation}           │
     │──────────────────────────────────►│  Verify, auth_level → 2
     │  AUTH DH response                 │
     │◄──────────────────────────────────│
     │                                   │
     │  === Encrypted channel ===        │
     │  UCP4 data (proto=3, BINME)       │
     │──────────────────────────────────►│
```

Auth messages use **msgpack** encoding. Post-auth data uses BINME+JSON.

Encryption

- Algorithm: NaCl `crypto_secretbox` (XSalsa20-Poly1305)
- Key: 32 bytes (Curve25519 DH shared secret)
- Nonce: 24 bytes (from 2-byte sequence number, zero-padded)
- Mandatory for all BLE communication (unlike WSS where TLS suffices)

Data Framing

```
[seq: 2 bytes BE] [proto: 1 byte] [payload...]

proto 0: ble-auth (msgpack)
proto 3: UCP4 data (BINME-wrapped JSON)
```

BLE-to-WiFi Bridge

The Protect application can also route BLE connections through an existing adopted device (bridge mode). The `bleSocket` handler creates a synthetic WebSocket connection with the same UCP4 protocol.

For Implementation

BLE provisioning is a secondary adoption path. For MVP, BLE advertisement for discovery is sufficient. The full GATT service with Curve25519 + NaCl + msgpack can be deferred.

---

16. State Persistence (NVS)

Namespace: `nvs_unifi`

Key	Type	Purpose
`wifi_ssid`	string	WiFi SSID
`wifi_password`	string	WiFi password
`c_uid`	string	Controller UID
`c_username`	string	Controller username

`c_password`	string	Controller password
`c_ip`	string	Controller IP
`c_port`	string	Controller port
`wss_info`	blob	WSS connection state
`reset_flag`	int	Factory reset flag
`host_%d`	blob	Host entries (indexed)
`m_password`	string	Device password (after changeUserPassword)
`certs`	blob (0x800)	Controller-provisioned certificate
`privkey`	blob (0x200)	CA private key material
`anon`	blob (16)	Anonymous identity (hardware RNG)

---

17. Audio / Speaker System

> ***Note:*** *This section is entirely from static firmware analysis. No audio playback has been implemented or tested. Hardware-specific details (TAS2560 register sequences, I2S pin assignments, 205-step init table) are from RE and may contain errors.*

Hardware

- **Amplifier**: TAS2560 over I2C (`i2c-0`) + I2S output
- **GPIO pins**: `tas2560_rst` (reset), `tas2560_intr` (interrupt)
- **I2S**: ESP32 I2S0, master TX, 44.1 KHz, 16-bit

I2S Configuration

```
Port:        I2S0, Master TX mode
Sample rate: 44100 Hz
Bit depth:   16-bit
Pins:        BCK=GPIO25, WS=GPIO26
DMA:         5 buffers x 512 bytes
```

Track Storage

- **10 tracks** (indices 0-9), validated by `track_num + 1 < 0x0b`
- **Path**: `/lfs/track/<track_no>/<filename>` (LittleFS)
- **Formats**: `.mp3` (type 0), `.pcm` (type 1); others rejected
- **Track data loaded into SPIRAM** via `heap_caps_malloc(size, MALLOC_CAP_SPIRAM)`
- **Default tracks** cannot be uploaded or removed
- **Duplicate detection**: MD5 hash comparison skips re-upload

Playback Chain

1. UCP4 `playSpeaker` → packed params `[track_no, volume, repeat, speaker]`
2. Load track from `/lfs/track/<N>/<filename>` into SPIRAM if not cached
3. Detect file type by extension
4. **MP3**: custom decoder (Layer III, MPEG1/2, mono/stereo/joint), resample to 44.1 KHz
5. **PCM**: direct I2S write passthrough
6. Volume: float multiplier `volume / 100.0` applied per sample
7. Repeat controlled by `repeat` parameter

TAS2560 Driver

- **Init**: 205-register sequence from firmware table
- **Mute/Unmute**: Register 0x07 (0x41=mute, 0x40=unmute)
- **Gain**: Register 0x04 lower nibble, 16 levels
- **Reset**: GPIO toggle

Buzzer (separate from speaker)

- Params: `volume` (<256), `freq` (4-8004 Hz), `duration` (<1001 ms)
- PWM-driven GPIO, not I2S

Upload Ringtone

- **Endpoint**: `PUT /api/uploadRingtone/*`
- Track number extracted from URL path
- Max content length: 10240 bytes (10 KB) — seems low; may be a per-chunk limit or a misread from RE
- Validates: not a default track, checks MD5 for duplicates, `.mp3` or `.pcm` only

For Reimplementation

Use ESPHome I2S media player or custom I2S driver. Replace the custom MP3 decoder with `libmad` or ESP-IDF `esp_audio`. Tracks on LittleFS can be replaced with SPIFFS. Buzzer is simple PWM — use ESPHome `ledc` output.

---

18. Key Constants

```
#define DEVICE_TYPE         "UP Chime"
#define DEVICE_MODEL        "up-chime"
#define FW_VERSION          "v1.7.20"
#define SYSID               0xF9EE
#define SYSID_FORMAT        "%04X"
#define WSS_PROTOCOL        "ucp4"
#define WSS_BUFFER_SIZE     5120
#define WSS_TIMEOUT_SEC     60
#define MAX_HOSTS           128
#define HOST_CYCLE_MOD      11
#define MAX_CONNECT_FAILS   60
#define RECONNECT_WAIT_MS   300001
#define CONNECT_WAIT_MS     3000
#define WIFI_TASK_STACK     7169
#define WIFI_TASK_PRIO      5
#define X_MODE              "0"
#define DEFAULT_USER        "ui"       // v2 cred version (0x03)
#define DEFAULT_PASS        "ui"       // v1 uses "ubnt"/"ubnt"
#define ADOPTED_STATE       0x0c
#define NVS_NAMESPACE       "nvs_unifi"
#define HTTPS_PORT          8080
#define DISCOVERY_PORT      10001
```

---

19. Implementation Checklist

Minimum Viable Device

- [ ] **UDP Discovery** — Listen on port 10001, respond with TLV packet
  - Header with correct data length in bytes 2-3
  - All required TLVs (0x0C, 0x0B, 0x01, 0x04, 0x10, 0x17, 0x03, 0x26, 0x2B)

  - Sysid in little-endian (TLV 0x10)
  - MGMT_IS_DEFAULT = non-zero when unadopted (TLV 0x17)
- [ ] **BLE Advertisement** — Manufacturer data with Ubiquiti company ID 0x04C5
- [ ] **HTTPS Server** (port **8080**) with factory ECDSA cert
  - `GET /api/info` — JSON response + `x-type` / `x-sysid` / `x-ident` headers
  - `POST /api/adopt` — Parse hosts/token/credentials from JSON body (NOT Basic auth), respond `{}`
  - `x-type` MUST be exact SKU string (e.g. `"UP Chime"`)
  - `x-ident` is bare MAC hex (e.g. `"AABBCCDDEEFF"` , NOT `"Chime-AABBCCDDEEFF"`)
- [ ] **WSS Client** — Connect to controller after adoption
  - All required headers (Sec-Websocket-Protocol, x-type, x-ident, x-token, etc.)
  - `x-type` must match SKU string
  - Factory cert as client cert, skip server cert verification
- [ ] **BINME Codec** — Encode/decode two-segment binary messages
- [ ] **UCP4 Handler** — Parse JSON commands, send responses
  - `getConsoleInfo` as first message after connect
  - Periodic status reports (~30s interval)
  - Handle at minimum: `getInfo` , `playSpeaker` , `reboot` , `factoryReset`

Key Pitfalls

1. **Discovery data length** — Bytes 2-3 of header MUST contain the TLV data length. A value of `0x0000` causes the console to ignore all TLV fields.

2. **Sysid byte order** — TLV 0x10 is read as `readUInt16LE()` . Send little-endian.

3. **x-type must be SKU** — The console resolves `modelKey` via `getType(x-type)` . It matches against exact SKU strings like `"UP Chime"` , `"UP Chime PoE"` . Sending `"Chime"` will fail.

4. **TLV 0x17 for adoption state** — `isManaged = !value` . Send non-zero (e.g. `0x01`) for an unadopted/adoptable device. Send `0x00` after adoption.

5. **Server cert verification** — The device must NOT verify the controller's TLS cert (set `skip_cert_common_name_check = true` , don't provide a CA cert for server verification). The controller's cert is not signed by the factory CA.

6. **BINME subtype byte** — Always `0x01` outgoing, never checked on parse. Do not set to `0x00` .

7. **WSS subprotocol MUST use native field** — Set `subprotocol = "ucp4"` via the WebSocket client's native subprotocol configuration field (e.g. `esp_websocket_client_config_t.subprotocol`). Do NOT include `Sec-Websocket-Protocol: ucp4` as a custom header string — this causes a 403 Forbidden from the controller (likely due to case-sensitivity or header merging issues). The firmware RE appeared to show it in custom headers, but live testing confirmed the native field is required.

8. **HTTPS port is 8080, NOT 443** — Protect's `getApiPort()` returns 8080 for all non-bridge devices. The controller will not find the device if serving on 443.

9. **Credentials are `ui` / `ui` for Chime** — The UP Chime reports credential version 0x03 (TLV 0x2C), which maps to v2 (`ui` / `ui`). Using `ubnt` / `ubnt` will fail.

10. **Adopt credentials are in JSON body** — The controller sends `username` / `password` in the POST body of `/api/adopt` , NOT via HTTP Basic auth headers.

11. **x-ident is bare MAC** — The `x-ident` WSS header and `/api/info` response header should be the bare MAC hex string (e.g. `AABBCCDDEEFF`), NOT the BLE name (e.g. `Chime-AABBCCDDEEFF`). The BLE prefix is only for BLE advertisement names.

12. **WiFi may not be ready at setup time** — On ESPHome with `AFTER_WIFI` priority, WiFi is started but not connected. The device IP will be `0.0.0.0` in early loop iterations. Wait for a valid IP before starting UCP4.

Hardware Tested

- ESP32 (any variant with WiFi + BLE)
- ESP-IDF v4.4+ (firmware uses v4.4.8, ESPHome uses v5.x — both work)
- ESPHome with `esp-idf` framework type

License and Disclaimer

This document and the accompanying source code are licensed under the PolyForm Noncommercial License 1.0.0 — free for hobbyists, researchers, and non-commercial use. Not licensed for commercial purposes.

Required Notice: Copyright 2026 Qoole (https://github.com/Qoole)

This project is not affiliated with or endorsed by Ubiquiti Inc. UniFi, UniFi Protect, and UP Chime are trademarks of Ubiquiti Inc. The factory TLS certificate referenced in this document was extracted from publicly available firmware. This project is intended for personal, educational, and research use only.

Document derived from reverse engineering UniFi Chime firmware v1.7.20 (sysid 0xF9EE), analysis of UniFi Protect application service.js, and live packet captures against UP Chime PoE hardware. March 2026.